Measuring Overheads in Distributed Deep Learning

Abdulfatah Bahbouh and Ishfaq Ahmad

***Abstract*—We measure how state sharding and expert routing affect memory use and execution time in distributed deep learning. Both add communication, but they change different quantities: sharding changes what each worker stores, and routing changes which worker processes each token. A throughput figure combines these added costs without identifying them. A 2.547-billion-parameter transformer runs under replicated data parallelism, ZeRO-2, and ZeRO-3 on Intel and NVIDIA GPU systems at one, two, and four nodes. ZeRO-2 partitions gradients and optimizer state; ZeRO-3 also partitions the parameters. A separate benchmark changes token destinations in an expert-parallel layer. Communication overlap is disabled, so each collective's cost is exposed, and inter-node transfers pass through host memory. Across the multi-node sharding configurations, ZeRO-2 reduces peak memory from about 58 GiB to 14–17 GiB per worker. It shortens the step at one node but lengthens it at four nodes in the measured runs. DeepSpeed's communication logger was enabled in every sharding run. It grows the step during execution and changes the apparent distribution of communication time. No logger-disabled run was collected, so the report does not establish ZeRO-3's execution time without that instrumentation. The four-node H100 comparison includes that logging overhead. Under skewed routing, median step time falls from 433.9 to 170.7 ms on the Intel configuration but rises from 15.6 to 28.6 ms on H100. The report also shows that an interval's label does not establish what work fills it. It documents the timer boundaries, collective counts, and transports behind every number, as the experimental record for a companion survey.**

***Index Terms*—distributed training, ZeRO, sharding, mixture of experts, performance measurement, communication overhead**

I. Introduction

We examine two sources of communication in distributed training. In replicated data-parallel training, each worker stores the model parameters, gradients, and optimizer state, and the workers exchange gradients at every step. State sharding divides this stored training state among the workers: ZeRO-2 partitions gradients and optimizer state, and ZeRO-3 also partitions the parameters [2], [3]. Sharded training must then gather or reduce the distributed state when it is used. Expert parallelism instead places different subnetworks, called experts, on different workers [4], [5]: each token must be sent to the worker holding its assigned expert, and the resulting outputs must be returned. Both designs create communication, but they change different quantities: sharding changes what each worker stores, and routing changes which worker processes each token.

Reducing memory use does not necessarily shorten a training step. The added communication takes time to transfer data, to start and complete each operation, and to copy data through host memory, and workers may wait for one another when their loads differ. A throughput figure combines these costs without identifying them. We therefore examine which operations account for step time and how their cost changes with the number of nodes and the communication path. The routing microbenchmark asks a separate question: with the number of tokens each worker sends held fixed, how does redistributing their destinations change exchange time and worker computation?

We disable communication overlap, so communication does not run behind computation. On every worker we time each collective interval, one group data exchange. We call the result exposed cost. The intervals cover the work between call issuance and completion, which may combine transfer, launch, queued work, and waiting. A decoder-only transformer with 2.547 billion parameters runs under replicated data parallelism, ZeRO-2, and ZeRO-3 on two GPU configurations at one, two, and four nodes. A separate microbenchmark times the token exchange of an expert-parallel layer under balanced and skewed routing. The measurements address three questions.

1) How does ZeRO-2 change peak memory and step time relative to replicated data parallelism, and how does the change differ between the two configurations?
2) How do collective operations, transport, and measurement instrumentation contribute to the observed ZeRO-3 step time?
3) How does skewed routing change the phase times of an expert-parallel exchange in the two configurations?

This report expands the case studies in Section 9 of the companion survey [1]. It documents the experimental methods, collective counts, communication-path checks, and raw records supporting those results. The survey provides the literature review and the diagnostic framework; this report provides the detailed experimental evidence. The measurements describe execution with communication overlap disabled and inter-node transfers through host memory, not production throughput. Section VI states the limits of these comparisons.

II. Experimental Methodology

A. Workloads and software

The sharding experiments use a workload large enough that replicated state dominates device memory: under the replicated baseline, labeled DDP, each rank holds the weights, gradients, and AdamW state of the 2.547-billion-parameter model (Table I).

A rank is one worker process driving one GPU, or one tile on the Intel system (a Max 1550 card exposes two tiles, so eight tiles are four cards). The number of ranks is the number of nodes times the ranks per node.

The authors are with the Department of Computer Science and Engineering, The University of Texas at Arlington, Arlington, TX, USA.

Technical report. This document is not peer reviewed. It accompanies a manuscript under review at ACM Computing Surveys [1].

TABLE I
EXPERIMENTAL CONFIGURATIONS.

Sharding experiments	
Model	decoder-only transformer, 2.547B parameters
Layers / hidden / heads	30 / 2560 / 20
Feed-forward	SwiGLU, intermediate dimension 6912
Vocabulary	32,768, untied embedding and head
Sequence	2048 tokens, one sequence per rank
Precision	BF16 compute, FP32 master state (AdamW)
Activation checkpointing	off
Engine	DeepSpeed 0.19.4; `overlap_comm=false`
Modes	stage 0 (replicated baseline, labeled DDP), ZeRO-2, ZeRO-3
PyTorch	2.11.0+cu128 (H100); 2.13.0+xpu (Intel)
Iterations	24–40 measured after 6–10 warm-up; 3 repetitions
Routing microbenchmark	
Expert	one SwiGLU feed-forward network per rank, hidden 2560, intermediate 6912
Data type	BF16 tokens, weights, and exchange
Tokens	4096 per rank per iteration
Ranks	16 over two Intel nodes; 8 over two H100 nodes
Routing	balanced: equal counts to every expert; skewed: half of every rank's tokens to expert 0, the rest spread over the other experts
Iterations	50 measured after 5 warm-up; 3 repetitions
Library	`torch.distributed` all-to-all; no DeepSpeed

Four collective operations name the exchanges in this report: an all-reduce combines every worker's contribution and returns the result to all of them; an all-gather collects the workers' pieces and gives every worker the full set; a reduce-scatter combines the contributions and leaves each worker one part; and an all-to-all sends a different piece from every worker to every other worker. Each rank processes one 2048-token sequence of synthetic tokens per step, so the work per rank is the same at every scale, and no input pipeline, checkpointing, or evaluation cost enters the step.

DeepSpeed runs all three sharding modes [3]. ZeRO defines stages 0 through 3, each partitioning more state; stage 1 is not used in these experiments. Its stage 0 serves as the replicated baseline (labeled DDP in figures and tables): a data-parallel mode that all-reduces bucketed gradients as DDP does [6]. ZeRO-2 and ZeRO-3 partition state as described in Section I [2]. The routing microbenchmark is a separate program with its own configuration, listed in the lower block of Table I.

B. Platforms

Table II lists the two configurations. They differ in accelerator, ranks per node, runtime, and transport, so the report compares how each configuration scales and which operations account for its step time, not absolute rates or vendors.

All inter-node transfers pass through host memory rather than moving directly between GPU memory and the network. The H100 experiments stage NCCL (NVIDIA's collective communication library) transfers over InfiniBand through pinned host memory. The Intel experiments use oneCCL (Intel's collective library) over TCP carried on the InfiniBand hardware, which also stages data in host memory; the fabric's native RDMA mode is not used there. The H100 transport control replaces host-staged InfiniBand with TCP sockets on the same nodes, changing only the transport. Direct GPU-to-network transfer was unavailable on the tested H100 system; the job logs record the failure and the transport NCCL selected, and Section VI-B gives the error and the settings.

TABLE II
THE TWO CONFIGURATIONS AND THE H100 TRANSPORT CONTROL.

Configuration	Ranks per node	Nodes	Communication path
Intel Max 1550	8 tiles	1, 2, 4	oneCCL over TCP, host-staged
NVIDIA H100	4 GPUs	1, 2, 4	NVLink within a node; NCCL over InfiniBand, host-staged
H100 control	4 GPUs	2	NCCL over TCP sockets

C. Timing method

Communication overlap is disabled (Table I), so each collective completes before the step continues. A barrier precedes each step, and device synchronizations separate the forward pass, the backward pass, and the optimizer step. Step time is measured from the end of the starting barrier to the end of the optimizer update. For each iteration we take the longest time across workers. A run's step time is the median over its measured iterations. We report the median of the three repetitions.

A collective interval starts when its call is issued and ends after the subsequent device synchronization returns; an asynchronous call is waited on before that synchronization. The interval can include queued device work as well as the collective itself (a kernel launched before the collective may still be draining when the interval opens), so it is not a measurement of transfer time alone.

Two timer boundaries appear in the sharding runs, and Table III states what each interval includes. Timers wrap `torch.distributed` calls, and in some H100 runs they instead wrap DeepSpeed's communication module; when a DeepSpeed call invokes a `torch.distributed` call, only the outer call is counted, so no interval is counted twice. Independently of these timers, DeepSpeed's own communication logger (`comms_logger` with `prof_all`) was enabled in every sharding run: it records a pair of device events for each call and synchronizes the device after it, and Section III-B shows that its cost grows during a run. Moving the timer boundary changes where that work is attributed, not whether it executes. Figure 1(a) shows the state each rank holds under each mode, example collective counts for this configuration, and the DeepSpeed-layer timed interval. Table III lists both timer boundaries. Figure 1(b) shows the two configurations and the phases of the routing microbenchmark.

Operation totals are sums over the measured iterations divided by their number. Figure 2(c) uses rank-mean interval sums and takes the residual against the median step. Table IV instead reports rank 0 of one repetition. The residual, labeled compute + unattributed, contains computation not already inside a timed interval, the optimizer arithmetic, host-side

TABLE III
WHAT EACH TIMED INTERVAL INCLUDES.

Interval	What it includes	How to read it
complete step	all work between the step boundaries, including logger work that executes there	total elapsed time of this instrumented configuration
DeepSpeed communication call	the underlying collective and the surrounding work inside that call, including logger work when this layer is timed	time attributed to the software call, not pure parameter transfer
underlying collective call	work from issuance through completion at the `torch.distributed` boundary; queued work can still be included	a narrower collective interval; logger work outside it remains in the step

framework work between calls, and logger work that falls outside the timed intervals.

Most cells, each a combination of configuration, mode, and node count, measured 40 iterations after 10 warm-up iterations; some Intel cells measured 30 after 8 or 24 after 6, and the artifact repository described in the closing section records the window of every launch. The window matters only where step time drifts during a run (Section III-B). Six Intel two-node runs made with an earlier harness version keep valid step times but no collective timings.

The measured quantities relate to the terms of the survey's step model [1] as follows:

- **Measured directly:** peak memory per rank, step time, the timed interval of each collective, and the number of collectives of each type per step.
- **Aggregated:** gradient reduction and parameter gathering, each the sum of the intervals of its collectives; the step, the maximum across ranks.
- **Not separately resolved:** transfer, launch, queueing, and waiting within a collective's interval; computation versus framework overhead within the residual; and the logger's cost, which falls inside the DeepSpeed-layer intervals when that layer is timed and inside the residual otherwise.

D. Routing microbenchmark

The second source of communication, expert routing, is isolated by a separate microbenchmark that times the token exchange of an expert-parallel layer [4], [5]. Each rank hosts one expert and sends 4096 tokens per iteration. Four phases are timed on every rank with device synchronization: the dispatch all-to-all, the expert's forward and backward computation, the return all-to-all, and the wait at the final barrier.

Balanced routing sends equal token counts to every expert. Skewed routing sends half of every rank's tokens to expert 0 and spreads the other half over the remaining experts, so expert 0 receives 8 times its balanced load on the 16-rank Intel configuration and 4 times on the 8-rank H100 configuration. Token assignments are fixed by the benchmark rather than learned, and experts accept all assigned tokens. The experiment therefore measures the specified routing patterns without a learned routing model or a token-capacity limit. The benchmark calls `torch.distributed` directly and does not use DeepSpeed or its logger.

III. STATE SHARDING

Figure 2 collects the sharding results. Panel (a) gives the median step time of DDP and ZeRO-2 on a log scale, with whiskers that span the three repetitions. Panel (b) gives peak memory per rank; the DDP curves of the two configurations coincide at the top. Panel (c) gives the composition of the four-node step: its communication categories are rank-mean interval sums, its residual is the difference from the median step, and the H100 ZeRO-3 shares come from runs timed at the DeepSpeed layer while the Intel shares come from `torch.distributed`-layer timers. Panel (d) plots the ZeRO-3 step time at every tenth measured iteration against the iteration index, which counts warm-up and measured iterations, with one line per run and a dotted line at iteration 11; the logger is enabled in every run shown. The series labeled IB (earlier job) is an earlier two-node H100 run with the same transport settings, lying on the same trend. The ZeRO-3 and H100 ZeRO-2 times include the logging overhead described in Section III-B.

A. ZeRO-2: memory and step time

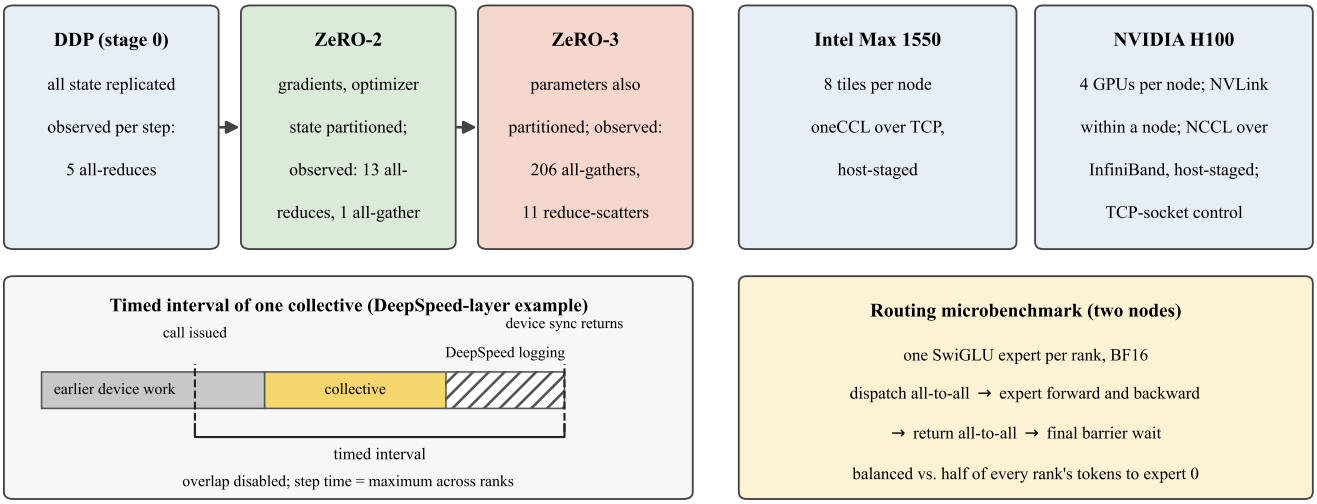
ZeRO-2 partitions gradients and optimizer state and keeps the parameters replicated. In this DeepSpeed configuration it reduces gradients with 13 bucketed all-reduces per step (buckets group gradient tensors into fewer messages), against 5 under the replicated baseline, and adds one all-gather that redistributes the updated parameters; the counts are read from the communication logger.

Memory. Across the multi-node configurations, ZeRO-2 reduces peak memory per rank from 57.8–57.9 GiB to 14.2–16.9 GiB, a reduction by a factor of 3.4–4.1, with activation checkpointing disabled (Fig. 2b). The replicated baseline already fits in these configurations, so the result is added memory headroom, not a change in whether the model fits.

Step time. At one node, ZeRO-2 reduces step time from 302.4 to 262.9 ms on H100 and from 802.2 to 601.0 ms on Intel. At four nodes, it increases step time by 11.9% on the H100 configuration, from 586.7 to 656.6 ms, and by 107.9% on the Intel configuration, from 3,718.1 to 7,731.8 ms. On Intel, parameter gathering takes 4,842 ms and gradient reduction 2,618 ms, together 96.5% of the step; under DDP, gradient reduction is already 80.7% of the four-node step.

Logging. The H100 ZeRO-2 step grew by 2.1 to 2.3 ms per iteration during measurement, while the replicated baseline, after its first measured iteration, grew by 0.3 to 0.45 ms (Section III-B). The four-node H100 difference therefore includes logging overhead that these records do not let us remove. On Intel the growth is about 1 ms per iteration at one node, small beside a four-node step of 7.7 s; the multi-node Intel steps vary too much from one iteration to the next to resolve it.

ZeRO-2 therefore reduces peak memory in both configurations, but its effect on step time differs between them. The configurations differ in accelerator, ranks per node, runtime, and transport, so this comparison does not isolate the effect of



(a) state, collectives observed in this configuration, and timer boundaries

(b) configurations and routing phases

Fig. 1. Measurement setting of the sharding runs (a) and the routing microbenchmark (b).

any one difference; the transport comparison of Section III-B changes only the transport.

B. ZeRO-3: parameter gathering

ZeRO-3 also partitions the parameters across ranks. It gathers each layer's parameters during the forward and backward passes, adding repeated parameter exchange to the step; gradients are still reduced, with 11 reduce-scatters per step in this configuration. On H100, peak memory per rank falls from 20.4 GiB at one node to 15.7 GiB at two nodes and 13.3 GiB at four.

1) Step time grows during every run: The ZeRO-3 step time rose steadily during measurement in every run, so any summary value depends on the measurement window. Figure 2(d) therefore plots it per iteration rather than as a median. The per-iteration times in the job logs rise by 0.54 to 0.56 s per step on H100, at one, two, and four nodes and over both transports, and by 0.27 to 0.29 s per step on Intel; at one H100 node the step was 7.9 to 8.1 s at the first measured iteration and 24.1 to 24.4 s at the 31st. A window's median therefore depends on where the window lies. Every H100 ZeRO-3 run measured 40 iterations after 10 warm-up iterations, so its medians, 18.33 s at one node, 18.86 s at two, and 19.16 s at four, sit at the same point of the same rising trend. The Intel runs did not share a window: the one-node runs measured 40 iterations after 10 warm-up iterations, the two-node runs 40 after 10 or 30 after 8, and the four-node runs 24 after 6. Their medians, 10.29, 11.46, and 9.74 s, come from different points of the trend and do not rank the node counts.

2) Why the step grows: logging overhead: The growth is consistent with the communication logger's implementation. After recording a new call, the logger revisits the event pairs accumulated for that operation, synchronizing on each pair [7]; its work therefore increases as the run proceeds. A fitted cost of about 13 µs per earlier call of the same operation reproduces the measured growth of all three modes on H100. The replicated baseline, ZeRO-2, and ZeRO-3 issue 5, 14, and 218 logged collectives per step. The fitted cost predicts growth of 0.3, 2.2, and 550 ms per iteration against 0.3 to 0.45, 2.1 to 2.3, and 541 to 561 ms measured. On Intel, a fitted 7 µs per earlier call reproduces the ZeRO-3 growth and the one-node ZeRO-2 growth. These coefficients are fitted to the observed growth, not measured separately, and the growth depends on neither node count nor transport. No run with the logger disabled was made, so ZeRO-3's step time without this instrumentation is not measured here.

3) Where the timer places the overhead: Table IV gives the per-iteration accounting of one repetition per node count, read from rank 0 in runs timed at both layers. Each step issues 145 parameter gathers through DeepSpeed's `allgather_fn`, 61 other all-gather calls, 11 reduce-scatters, and one all-reduce, the same at every node count. The timed operations sum to 99.6, 99.9, and 99.7% of these repetitions' steps, and the gathers alone to 99.1, 98.6, and 98.4%. The reduce-scatter total is 75 ms at one node and 221 and 243 ms at two and four.

The timer layer changes this picture. Five two-node runs timed only `torch.distributed` calls: three over InfiniBand and the first two of the three TCP repetitions. In those runs the same 206 gathers total 0.7 to 1.1 s per step over InfiniBand and 1.9 to 2.4 s over TCP. The steps themselves are unchanged, 18.8 to 19.0 s and 21.0 to 21.1 s. Most of the time that Table IV assigns to gathers is therefore spent inside DeepSpeed's communication layer rather than in the `torch.distributed` collectives. The logger was enabled in these runs as well. Moving the timer boundary moves its work between the gather intervals and the residual. That movement is how an 18 to 19 s step can contain about 1 s of `torch.distributed`-level collectives over two-node InfiniBand. The Intel runs, also timed only at that layer, show the same pattern: the gathers account for 8 to 28% of the median step, more at more nodes (Fig. 2c), and the residual holds most of the rest.

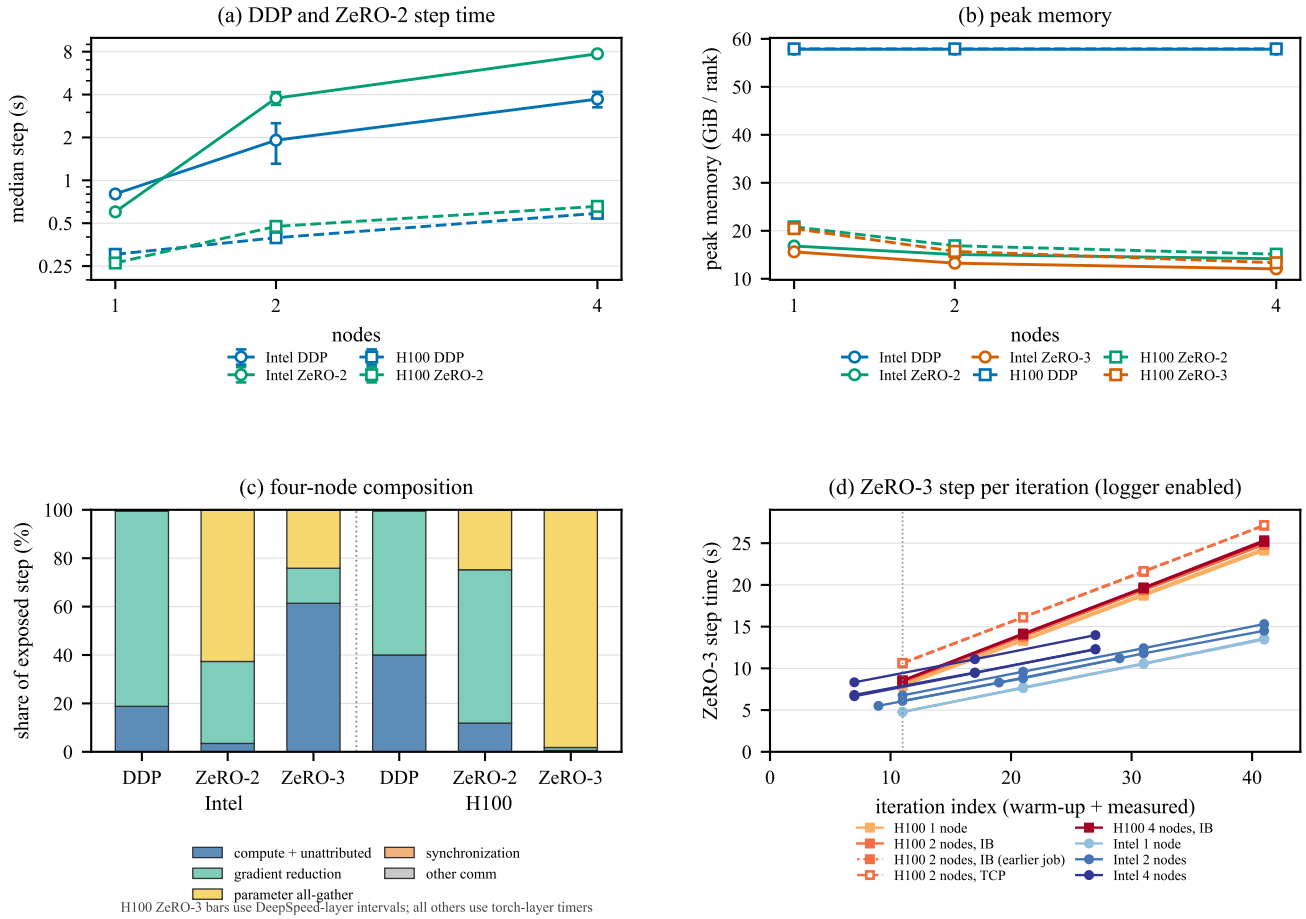


Fig. 2. State sharding: step time (a), peak memory (b), step composition (c), and the ZeRO-3 step over training (d).

TABLE IV
PER-ITERATION ZERO-3 COLLECTIVE COUNTS AND TIMES ON H100: RANK 0 OF ONE REPETITION PER NODE COUNT, DEEPSPEED-LAYER TIMERS.

Operation	Count per step	1 node ms	2 nodes ms	4 nodes ms
`allgather_fn` calls	145	12,613	12,979	13,174
other all-gather calls	61	5,408	5,521	5,564
reduce-scatter	11	75	221	243
all-reduce	1	2.5	2.0	2.0
barrier	1	1.3	4.7	6.2
measured step		18,181	18,755	19,052

The barrier interval precedes the step boundary of Section II-C and is listed for completeness; the 218 logged collectives are the gathers, the reduce-scatters, and the all-reduce.

4) Matched iterations, one node, and the transport: Because the Intel windows differ in length, we compare node counts at the same training iteration, the eleventh, the earliest iteration inside every run's measured window (the dotted line in Fig. 2d). There the Intel step is 4.78 s at one node, 6.10 s at two, and 7.89 s at four, and the H100 step is 7.93, 8.37, and 8.52 s (medians of the repetitions, interpolated between logged iterations where iteration 11 was not logged). Comparing at the same iteration controls for the amount of preceding measurement; it does not measure the logger's cost separately. At that matched point, the measured step lengthens with node count on both configurations. The replicated baseline at the same node counts takes 0.302, 0.395, and 0.587 s on H100 and 0.80, 1.91, and 3.72 s on Intel.

Two observations constrain the explanation of the large ZeRO-3 steps. First, the one-node H100 run has no inter-node traffic, yet at iteration 11 it is only 0.4 to 0.6 s shorter than the multi-node InfiniBand runs and it grows at nearly the same rate (0.54 against 0.55 to 0.56 s per step), so inter-node communication is not required for the large step time. Second, at two nodes, replacing host-staged InfiniBand with TCP sockets adds 2.0 to 2.3 s to the step at every logged iteration, which raises the 40-iteration median from 18.86 to 21.10 s. In the runs timed at the `torch.distributed` layer, the time of the collectives themselves rises by a similar amount, from 0.9 to 1.2 s per step to 3.0 to 3.4 s. The transport therefore changes the collectives' time, by about 2 s per step, but not the growth. The large step times persist in the one-node run and under both transports, so inter-node bandwidth alone cannot explain them; these observations do not separately quantify data movement, dispatch, launch, queueing, or synchronization inside the collectives.

Because the logging cost grows, the ratio of the ZeRO-3 step to the replicated baseline changes with the observation window, and no single ratio describes this instrumented config-

uration. With prefetching, overlap, and a direct GPU transport, published deployments report sharded training throughput approaching the replicated baseline [2], [8].

IV. Expert Routing

Expert routing is the second source of communication in this report. Its benchmark changes token destinations while keeping the number of tokens each worker sends fixed. Skew shortens the Intel step from 433.9 to 170.7 ms and lengthens the H100 step from 15.6 to 28.6 ms (Fig. 3); the phase timings show where those differences appear.

The phase bars summarize one selected worker, and a different worker may be selected in each condition; the step percentiles summarize complete iterations across all workers. Figure 3 and Table V report, for each condition, the selected worker: the rank with the largest sum of phase medians. A rank's phase median is the median over the three repetitions of its per-repetition median phase time. The selected worker is therefore a summary statistic, not necessarily the slowest worker on every iteration. The tick and whisker mark the p50 and p95 step times, the 50th and 95th percentiles of per-iteration step time, where an iteration's step time is the maximum across workers and the percentiles are medians over the three repetitions. A stacked bar therefore summarizes phase medians rather than one measured iteration; its total differs from the median step time by at most 7%. The right panel of Fig. 3 shows a different comparison: every rank's expert computation time under skew relative to the median rank, where rank 0 hosts expert 0 and takes 12.6 times the median on Intel and 6.4 times on H100.

Intel, 16 ranks. The Intel result is a reduction in complete-step time despite more work at expert 0. Balanced routing takes 433.9 ms per step at the median, with the dispatch and return exchanges accounting for 98% of the selected worker's phase-time total (Table V). Under skew the median step falls to 170.7 ms, although expert 0, hosted by rank 0, takes 57.5 ms to compute, 12.6 times the median rank's expert time. Each rank sends 4096 tokens in both conditions; only their destinations change. The exchange intervals become shorter, and the measurements do not establish why; network contention is a possible explanation, but the study did not collect the link-level counters needed to test it.

The selected worker differs between the Intel conditions: rank 3 under balanced routing and rank 13 under skewed routing, with dispatch medians of 233.8 and 18.0 ms. Under skew, rank 13's expert-compute interval is 4.6 ms, while expert 0, hosted by rank 0, takes 57.5 ms; the values describe different workers. Rank 13's return exchange takes 145.3 ms; because an all-to-all completes only when every rank has sent its part, that interval can include waiting for expert 0's output as well as transferring data, and the benchmark does not separate those costs.

H100, 8 ranks. Skew increases the median step from 15.6 to 28.6 ms, and the added time appears in the selected worker's return exchange, which grows from 6.5 to 22.4 ms. Under skew, expert 0 processes 16,384 tokens, four times as many as an expert receives under balanced routing, and takes 9.39 ms to compute against 1.48 ms for the median rank: a 6.4-fold difference in computation time for a 7.0-fold difference in assigned token count (expert 0 receives 16,384 tokens and each other expert about 2,341). The selected worker's final barrier interval is small under skew (0.2 ms) while its return-exchange interval is large; the return interval can include waiting for expert 0's output, but the timers do not separate that wait from transfer time. Final barrier waits stay below 7 ms on every rank.

TABLE V
Selected-worker phase medians and whole-step percentiles (ms) for the four routing conditions.

			selected worker's phase medians				whole step	
Configuration	Routing	worker	dispatch	expert	return	wait	p50	p95
Intel	balanced	3	233.8	7.0	200.4	0.9	433.9	496.3
Intel	skewed	13	18.0	4.6	145.3	13.7	170.7	195.9
H100	balanced	4	5.2	2.5	6.5	1.7	15.6	16.8
H100	skewed	4	4.7	1.5	22.4	0.2	28.6	30.0

Skewed routing therefore redistributes time among the phases differently in the two configurations: it shortens the exchange-dominated Intel step and lengthens the H100 step through the return exchange. In both configurations the exchange intervals remain the largest phase. The configurations differ in the number of ranks, the transport, and the relative skew (8 against 4 times the balanced load on expert 0), so the contrast shows that the effect of skew depends on the configuration without isolating any one cause. Production expert-parallel systems address both the exchange and the load balance [9], [10].

V. Summary of Findings

The measurements separate memory savings from execution-time improvement. They also show that an interval's label does not establish what work fills it. ZeRO-2 reduces memory per worker, but its measured step-time effect changes with node count. ZeRO-3's logging overhead grows during a run and can dominate the intervals labeled as communication. Routing skew holds the number of tokens each worker sends fixed while changing the number a single expert processes. It shortens both Intel exchanges and lengthens the H100 return exchange. In both configurations the exchange intervals remain the largest measured phases. These results describe the tested instrumented paths and support the interpretation of the survey's case studies. None of the comparisons isolates a change in the limiting component, which is the survey's definition of bottleneck migration. Table VI summarizes the outcomes and what the records leave unresolved.

VI. Limitations and Reproducibility

A. Scope of the results

The workloads use fixed synthetic inputs, so the results describe memory use and execution time, not accuracy or convergence. Communication overlap is disabled, and inter-node transfers use host memory, so the measurements do not

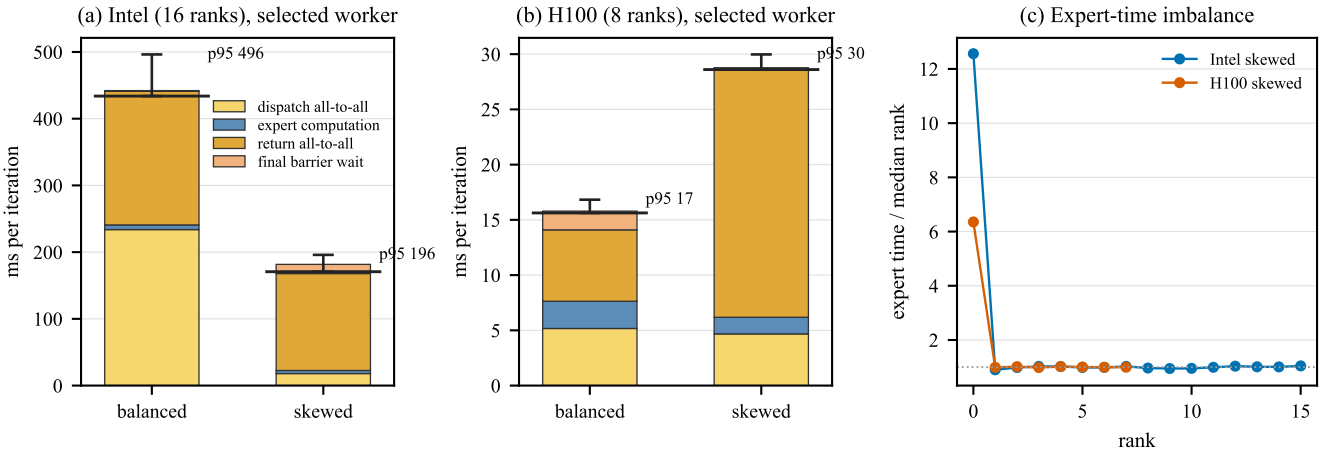


Fig. 3. Expert routing. (a) Intel: the selected worker's phase medians under balanced and skewed routing. (b) H100: the same. (c) Each rank's expert computation time under skew, relative to the median rank.

TABLE VI
SUMMARY OF THE MEASURED COMPARISONS.

Comparison	Change	Measured outcome	Unresolved question
ZeRO-2: memory	gradients and optimizer state partitioned; 13 all-reduces and 1 all-gather per step, against 5	peak memory falls 3.4–4.1×, from about 58 to 14–17 GiB per worker	–
ZeRO-2: step time	same runs as above	one node: faster on both; four nodes: +11.9% on H100 (includes logging) and +107.9% on Intel	which configuration difference sets the four-node gap
ZeRO-3: measurement overhead	parameters also partitioned; 206 all-gathers and 11 reduce-scatters per step, logger enabled	step grows 0.54–0.56 s per iteration on H100 and 0.27–0.29 s on Intel; at matched iteration 11 the step lengthens with node count; `torch`-level collectives about 1 s per step over two-node H100 InfiniBand	ZeRO-3's step time without the logger
Routing, Intel (16 ranks)	balanced to skewed; expert 0 at 8× its balanced load	step 433.9 to 170.7 ms; selected worker's dispatch 233.8 to 18.0 ms; exchanges remain the largest phase	why the exchanges shortened
Routing, H100 (8 ranks)	balanced to skewed; expert 0 at 4× its balanced load	step 15.6 to 28.6 ms; selected worker's return exchange 6.5 to 22.4 ms	transfer versus waiting within the return exchange

estimate throughput with overlap or direct GPU-to-network transfers enabled. Communication logging adds overhead in the sharding runs, including the within-run growth observed for ZeRO-3 on both systems; Section III-B reports the observed growth and its fitted explanation. No logger-disabled baseline was collected. The routing microbenchmark does not use this logger. Because the Intel ZeRO-3 cells used windows of 24 to 40 measured iterations, their medians are not comparable across node counts; Section III-B compares them at a matched training step.

The H100 transfer failure occurred on the tested system and does not establish a general limitation of NCCL over InfiniBand. The two configurations differ in accelerator, ranks per node, runtime, and transport, and the routing comparison also differs in the load on expert 0 relative to balanced routing. These differences prevent attributing the observed contrasts to the accelerator or the transport alone.

Each experimental configuration was repeated three times within one or two Slurm allocations. We report medians and ranges, not confidence intervals. Half the observed range is at most 0.6% of the median in the H100 experiments and 0.2% in the single-node Intel experiments. In the multi-node Intel experiments it ranges from 4% to 32%. For two-node ZeRO-3, most of the range comes from one repetition's shorter window. The largest variation occurs for two-node DDP, whose repetitions take 1765, 1912, and 2974 ms. Multi-node Intel comparisons should therefore emphasize broad differences rather than small percentage changes.

Table VII maps the disclosure checklist of the companion survey [1] to this report.

B. Environment

Both configurations run Python 3.12 with only the cluster Python module loaded; the Intel runs fail if MPI or compiler modules add a second copy of oneCCL or libfabric. The H100 runs use PyTorch 2.11.0+cu128 with

TABLE VII
THE SURVEY'S DISCLOSURE CHECKLIST, APPLIED TO THIS REPORT.

Checklist area	In this report
Infrastructure	interconnects and transports in Table II; no direct GPU transport on any multi-node run
Timing and phase attribution	exposed timing with overlap disabled; two timer boundaries (Table III); rank-mean and rank-0 aggregations stated per table; transfer, launch, queueing, and waiting within a collective not resolved; logging overhead identified but not removed; medians for sharding, p50 and p95 for routing; no p99
Communication settings	collective types and counts per step (Section III); reduce bucket 2.5×10^8 elements; ZeRO-3 prefetch bucket 5×10^8 and persistence threshold 10^5; no compression
Convergence evidence	outside the scope: synthetic inputs, no accuracy or time to a target

NCCL 2.28.9. Every launch of the first measurement job that let NCCL transfer directly from GPU memory failed with an `IBV_WC_LOC_PROT_ERR` protection error, so the runs set `NCCL_NET_GDR_LEVEL=LOC` to stage inter-node transfers through host memory, and the TCP control adds `NCCL_IB_DISABLE=1` [11].

The Intel runs use PyTorch 2.13.0+xpu with the XCCL backend over oneCCL. They use TCP through the InfiniBand interface with `FI_PROVIDER=tcp`. The libfabric installation in the virtual environment is selected through `FI_PROVIDER_PATH`, and `CCL_ATL_HMEM=0` is set. The launch scripts configure these variables and clean up PMI environment variables for each rank, and the recorded package versions for both configurations are stored in the `environment/` directory.

The transfer failure is specific to the tested system rather than to NCCL over InfiniBand in general: on a system whose node image loads the GPU peer-memory module, NCCL ran every inter-node channel over GPUDirect RDMA [12].

C. Code and data

All code and data are in one repository, available at https://github.com/abooodbah/diagnosing-bottlenecks-and-overheads-in-parallel-and-distributed-deep-learning. It contains the measurement code, launch scripts, raw run records, and processed results. Job logs and recorded package versions document the execution environments; the job logs also hold the per-iteration step times of Section III-B. The harness changed during the campaign; the repository keeps every version that ran, the files recovered from the cluster, the evidence that ties each launch to its version, and the warm-up and measurement window of every launch. Table VIII links each main result to its supporting files. The script `scripts/reproduce_all.sh` regenerates every table and figure from the raw records and job logs and verifies checksums.

TABLE VIII
RESULTS AND THEIR SUPPORTING FILES (PATHS RELATIVE TO THE REPOSITORY).

Result	Supporting files
ZeRO-2 memory and step times	raw/*/*_s0_*, raw/*/*_s2_*
ZeRO-3 step times and collective timings	raw/*/*_s3_*; `comms` fields
Collective counts (145 + 61)	comms_per_rank0 in raw/h100/h100_n1_direct_s3_r2.json
Per-iteration step times	transport_logs/job_logs/*.out; processed/drift_launches.csv
Windows and harness version of each launch	processed/launch_windows.csv; provenance/
Logger cost model, Table IV	scripts/logger_model.py; scripts/table3.py
H100 transport comparison	raw/h100/*tcp-sockets*
GPUDirect failure	transport_logs/job_logs/h100_c.3476714.out
Routing phase results	raw/*/*moe*
Processed tables	scripts/analyze.py; processed/summary.csv

REFERENCES

[1] A. Bahbouh and I. Ahmad, "Diagnosing bottlenecks and overheads in parallel and distributed deep learning: A survey," 2026, manuscript under review, ACM Computing Surveys.

[2] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020, pp. 1–16.

[3] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2020, pp. 3505–3506.

[4] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," *International Conference on Learning Representations (ICLR)*, 2021.

[5] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[6] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "PyTorch distributed: Experiences on accelerating data parallel training," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.

[7] DeepSpeed contributors, "DeepSpeed 0.19.4 source code, files comm/comm.py and utils/timer.py," https://github.com/deepspeedai/DeepSpeed, accessed September 22, 2026.

[8] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li, "PyTorch FSDP: Experiences on scaling fully sharded data parallel," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3848–3860, 2023.

[9] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," *International Conference on Machine Learning (ICML)*, 2022, pp. 18 332–18 346.

[10] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram, H. Chau, P. Cheng, F. Yang, M. Yang, and Y. Xiong, "Tutel: Adaptive mixture-of-experts at scale," *Conference on Machine Learning and Systems (MLSys)*, 2023.

[11] NVIDIA, "NVIDIA collective communication library (NCCL) documentation, version 2.30.7," https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/, accessed July 22, 2026.

[12] A. Bahbouh and I. Ahmad, "An empirical systems analysis of gradient compression in distributed deep learning," 2026, manuscript in preparation.